



Tribhuvan University

Faculty of Humanities and Social Sciences

A PROJECT REPORT

ON

CHESS BOT USING NEGAMAX AND ALPHABETA PRUNING

Submitted to

Department of Computer Application

Aadim National College

In partial fulfilment of the requirements for the Bachelor's in Computer Application

Submitted by

Utsab Dahal

Symbol No: 105902087

August 2025

Under the Supervision of

Mr. Saroj Bhandari



Tribhuvan University
Faculty of Humanities and Social Science
Aadim National College

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Utsab Dahal** entitled “**CHESS BOT USING NEGAMAX AND ALPHABETA PRUNING**” in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

SIGNATURE

Mr. Saroj Bhandari

Supervisor

Aadim National College

Chabahil, Kathmandu



Tribhuvan University
Faculty of Humanities and Social Science
Aadim National College

LETTER OF APPROVAL

This is to certify that this project prepared by **Utsab Dahal** entitled “**CHESS BOT USING NEGAMAX AND ALPHA BETA PRUNING**” in partial fulfillment of the requirements for the degree of Bachelor’s in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

..... Mr. Saroj Bhandari Supervisor Aadim National College Chabahil, Kathmandu	 Mr. Yogendra Bahadur Mahata Coordinator Aadim National College Chabahil, Kathmandu
..... Internal Examiner	 External Examiner

Abstract

Chess Bot using Negamax and AlphaBeta Pruning is a web-based chess system that allows users to play against a computer-controlled opponent using the Negamax algorithm optimized with AlphaBeta Pruning. The system is designed to deliver efficient move evaluation and responsiveness using bitboards, Zobrist hashing, and transposition tables. It features a modern software architecture with a front-end developed using Angular, Chess.js, and Angular Material, and a back-end built with Spring Boot and PostgreSQL. Communication between the client and server is handled via RESTful APIs, while role-based authentication using Spring Security ensures secure user access. UCHES currently supports a player-versus-bot mode with adjustable difficulty levels, offering a scalable and interactive gameplay experience. This report outlines the design, implementation and testing processes followed during development, incorporating best practices in software engineering. The project demonstrates the integration of automated gameplay with robust web technologies to deliver a responsive and secure chess platform.

Keywords: *Chess bot, Negamax, Alpha-Beta pruning, bitboards, Zobrist hashing, Angular, Spring Boot, REST API, NNUE*

Acknowledgments

I express my sincere gratitude to everyone who contributed to the development of the "Chess Bot using Negamax and AlphaBeta Pruning" project. First, I would like to thank our project supervisor, **Saroj Bhandari** Sir, for his invaluable guidance, support, and encouragement throughout this project. His expertise and insights were instrumental in shaping the direction and scope of this project.

Furthermore, I extend my appreciation to our Coordinator **Mr. Yogendra Bahadur Mahata** Sir for providing the resources and support necessary to complete this project. The access to facilities and technology greatly facilitated our work.

Finally, I am thankful to my family and friends for their unwavering support and encouragement throughout this journey. Their belief in our abilities kept us motivated and focused.

This project would not have been possible without the contributions of all of these individuals and organizations. Thank you for your support.

Sincerely

Utsab Dahal

(105902087)

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
JWT	JSON Web Token
NNUE	Neural Network-based Evaluation
REST	Representational State Transfer
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

Supervisor’s Recommendation	i
Letter of Approval	ii
Abstract	iii
Acknowledgement	iv
Abbreviations	v
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope and Limitation	2
1.4.1 Scope	2
1.4.2 Limitations	2
1.5 Development Methodology.....	2
1.6 Report Organization	3
2 Background Study and Literature Review	4
2.1 Background Study	4
2.2 Literature Review	4
3 System Analysis and Design	6
3.1 System Analysis.....	6
3.1.1 Requirement Analysis	6
3.1.2 Feasibility Analysis	9
3.1.3 Object Modelling: Object & Class Diagram.....	11
3.1.4 Dynamic Modelling: State & Sequence Diagram	11
3.1.5 Process Modelling: Activity Diagram.....	13
3.2 System Design	15
3.2.1 Refinement of Classes and Object.....	15
3.2.2 Component Diagram	15
3.2.3 Deployment Diagram	15
3.2.4 Algorithm Details	16
3.3 Gantt Chart	17
4 Implementation and Testing	18
4.1 Implementation	18

4.1.1	Tools Used	18
4.1.2	Implementation Details of Modules	18
4.2	Testing	19
4.2.1	Test Cases for Unit Testing	19
4.2.2	Test Cases for System Testing	20
5	Conclusion and Future Recommendations	21
5.1	Conclusion	21
5.2	Lesson Learnt/Outcome	21
5.3	Future Recommendations.....	22
	Appendices	23
A	Screenshots	23
A.1	Screenshots	23
A.1.1	Chessboard Interface	23
A.1.2	Login Page.....	23
A.1.3	Game History Page.....	24
B	Code Snippets	25
B.1	Code Snippets	25
B.1.1	Negamax with AlphaBetaPruning.....	25
B.1.2	Angular Chessboard Component	27
B.1.3	Spring Boot REST Controller	29
	References	30

LIST OF FIGURES

1.1	Waterfall model	2
3.1	Use Case Diagram for UCHESS	6
3.2	Class Diagram for UCHESS.....	11
3.3	State Diagram for Game	12
3.4	Sequence Diagram	13
3.5	Activity Diagram for Game Flow	14
3.6	Component Diagram for UCHESS	15
3.7	Deployment Diagram for UCHESS	16
3.8	Project Timeline.....	17
A.1	Chessboard Interface of UCHESS	23
A.2	Login Page of UCHESS	24
A.3	Game History Page of UCHESS	24

LIST OF TABLES

3.1	Use Case Scenario for Playing Against Chess Bot	7
3.2	Use Case Scenario for Managing Game via External System	8
3.3	Use Case Scenario for Viewing Game Status	9
4.1	Tools and Technologies Used in UCHESS Project	18
4.2	Unit Test Cases	19
4.3	System Test Cases	20

Chapter 1: Introduction

1.1 Introduction

UCHESS is a web-based chess bot designed to provide an interactive and challenging chess-playing experience for users. The system allows players to compete against the computer opponent in a player vs. bot mode, using the Negamax algorithm with AlphaBeta Pruning for efficient move evaluation. To optimize performance, UCHESS uses bitboards for board representation, bit shifting for move generation, Zobrist hashing for position identification, and transposition tables for caching evaluations. The frontend is developed using Angular, with Chess.js handling chess rules and Angular Material ensuring a responsive and intuitive user interface. The backend, built with Spring Boot and PostgreSQL, for efficient data management and communication via REST APIs. Role-based authentication, implemented using Spring Security, ensures secure user access. This project aims to deliver a scalable and accessible chess platform, with plans for future enhancements like NNUE, game analysis and additional game modes.

1.2 Problem Statement

Traditional chess engines, such as those used in professional settings, often require significant computational resources, making them less suitable for casual players on standard hardware. Many web-based chess applications lack customizable difficulty levels or efficient move evaluation, resulting in either overly simplistic or excessively challenging gameplay. Additionally, existing platforms may not integrate modern web technologies for seamless user experiences or secure access control. UCHESS addresses these challenges by developing an efficient chess bot with adjustable difficulty (starting at depth 2), a responsive Angular-based interface, and secure role-based authentication, tailored for both novice and intermediate players.

1.3 Objectives

- To develop a chess bot system that plays against human users, employing the Minimax algorithm with Alpha-Beta Pruning and incorporating move validation and game rule enforcement.
- To implement optimized data structures, including bitboards, Zobrist hashing, and transposition tables, to enhance performance.
- To develop an interactive interface for players to engage with the chess bot.

1.4 Scope and Limitation

1.4.1 Scope

- Utilizes performance-optimized data structures such as bitboards and transposition tables.
- Built using modern web technologies: Angular (frontend), Spring Boot (backend), and PostgreSQL (database).
- Provides secure, role-based authentication for controlled access.

1.4.2 Limitations

- Currently limited to web-based interaction; no mobile or desktop application available.
- Chess Bot with Negamax and AlphaBeta Pruning Does not yet include advanced evaluation methods such as neural networks (NNUE).
- Multiplayer is not supported in the current designed system.

1.5 Development Methodology

The system was developed using the Waterfall model, a sequential methodology comprising distinct phases: requirements analysis, system design, implementation, and testing. Each phase was completed systematically before proceeding to the next, ensuring thorough planning and clear documentation throughout the development process, frequently.

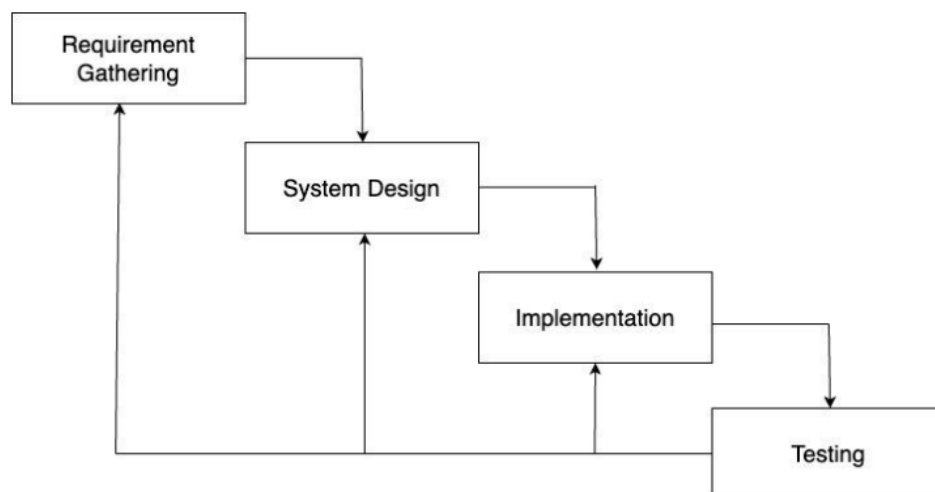


Figure 1.1: Waterfall model

1.6 Report Organization

The report is organized into five main chapters, each addressing different aspects of the chess bot with the Negamax and AlphaBeta pruning project.

Chapter 1: Introduction provides an overview of the project, including the background, the problem statement, and the objectives of the application. It also outlines the scope and limitations of the project, the development methodology employed (Waterfall Model), and the organization of the report.

Chapter 2: Background Study and Literature Review discusses foundational theories and related work on chess engines and web technologies.

Chapter 3: System Analysis and Design details the requirements, feasibility, and design of the system, including UML diagrams.

Chapter 4: Implementation and Testing covers the tools, modules, and testing strategies used in development.

Chapter 5: Conclusion and future recommendations summarizes the results and proposes future enhancements.

Chapter 2:Background Study and Literature Review

2.1 Background Study

Chess engines have evolved significantly, progressing from rudimentary rule-based systems to sophisticated tools capable of competing with human grandmasters. At their core, these engines rely on efficient board position evaluation and extensive game tree exploration to identify optimal moves within strict time constraints. Parallel search techniques are pivotal, leveraging multi-core processors to explore multiple move sequences concurrently. These techniques include systematic resource allocation across threads or adaptive task distribution based on computational resources, with performance varying by hardware configuration [1].

A critical component enhancing search efficiency is the bitboard data structure, which encodes the chessboard state using 64-bit integers. Bitboards facilitate rapid computation of piece movements, attacks, and positional evaluations through bitwise operations, making them well-suited for parallel processing. Studies, such as those by Rasmussen [2], demonstrate significant performance gains with bitboards in parallel environments, though results vary based on processor count and algorithm configuration. Move generation and validation ensure only legal moves are considered, while evaluation functions assess position strength through metrics like material balance, piece mobility, and strategic positioning [3]. These elements—parallel search, bitboards, and evaluation—are fundamental to designing a chess engine that balances computational speed, accuracy, and adaptability across diverse platforms.

2.2 Literature Review

The development of modern chess engines has been driven by advancements in algorithmic efficiency and optimized data structures. Kothari et al. [3] developed a chess engine emphasizing algorithmic efficiency without machine learning, focusing on optimized search algorithms and move generation. Their evaluation through benchmarks and gameplay demonstrates the efficacy of algorithm-driven approaches in resource-constrained environments. This aligns with the design goals of the UCHESS engine, which prioritizes lightweight, accessible performance, providing a foundation for enhancing its move evaluation and generation components.

Rasmussen [2] investigated the impact of bitboards on parallel search performance, implementing a chess program that achieved speedups of 9.4 on 12 processors and 9.2 on 22 processors. These results highlight bitboards as an effective data structure for parallel chess engines, though variability in performance suggests that factors such as thread allocation and hardware configuration warrant further exploration. For UCHESS, this underscores the importance of optimizing bitboard operations within its Bitboard class to enhance state management and search efficiency.

Säfström and Yllmark [1] conducted a comparative study of parallel search techniques, specifically Young Brothers Wait Concept (YBWC) and Lazy Symmetric Multiprocessing (SMP), using the open-source chess engine Crafty. Their experiments, involving 30 positions searched 50 times each and 30 matches against varied opponents, revealed that YBWC excels in resource-rich environments due to its structured resource allocation, while Lazy SMP performs better in resource-constrained settings due to its flexible approach. The lack of prior head-to-head comparisons of these techniques highlights the novelty of their findings, which are relevant for UCHESS's parallel search strategy selection.

These studies collectively inform the design of UCHESS, emphasizing efficient algorithms, bitboard optimization, and parallel search strategies. The variability in bitboard performance [2] and the limited comparison of search techniques [1] present opportunities for UCHESS to innovate through hybrid search approaches or targeted optimizations, potentially validated through extensive testing against diverse opponents.

Chapter 3: System Analysis and Design

3.1 System Analysis

This chapter discusses the proposed model of the system, the methodology used in building the system, tools and techniques, requirement analysis, and requirement specification. It also talks about the functional requirements of the system, the system design. The application architecture of the system will also be shown in this chapter, the use case diagram, data design, class diagram, state and sequence diagram, component diagram, and user interface design will all be shown in this chapter.

3.1.1 Requirement Analysis

The following section presents the complete set of functional requirements depicted by use case diagram and scenarios, as well as the non-functional requirements of UCHESS.

Functional Requirements

The functional requirements of a system define what a function should do, these are the desired operations of a program or software. This part of the report specifies the functional features compiled by the UCHESS.

- Allow users to play against a chess bot with adjustable difficulty levels.
- Implement standard chess rules using Chess.js for move validation and game state management.
- Provide role-based authentication for secure user login and session management.
- Enable communication between the frontend and backend via REST APIs.

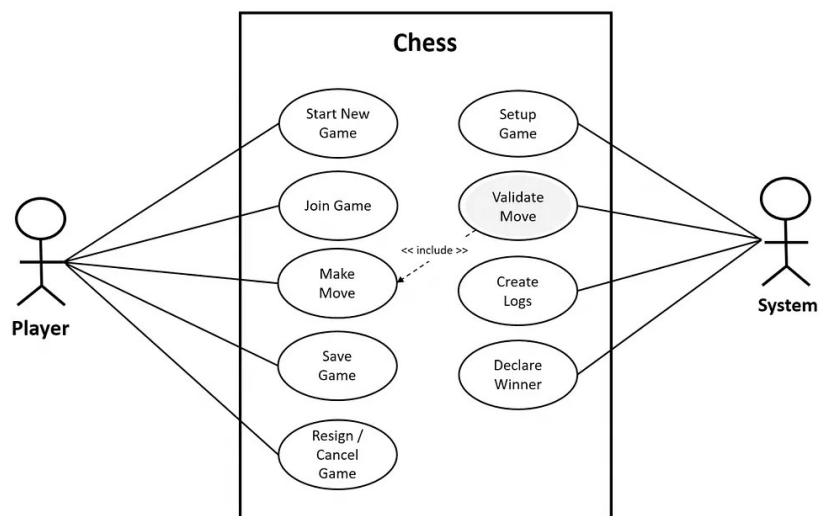


Figure 3.1: Use Case Diagram for UCHESS

The use case diagram (Figure 3.1) illustrates interactions, including game initiation, move execution, and bot response.

Table 3.1: Use Case Scenario for Playing Against Chess Bot

Use-Case Identifier	UC1: Play Against Chess Bot
Primary Actor	User
Secondary Actor	Bot
Description	User selects bot difficulty (e.g., Easy, Medium, Hard) and plays a chess game against the bot via the web interface
Pre Condition	App is running, user is logged in, game mode selected
Success Scenario	Game starts, user and bot take turns, game ends with win/draw/checkmate message displayed
Failure Scenario	Game crashes, bot move computation fails, or move validation errors occur

Table 3.2: Use Case Scenario for Managing Game via External System

Use-Case Identifier	UC2: Manage Game via External System
Primary Actor	External System
Secondary Actor	X
Description	External system (e.g., chess client) interacts with UCHESS via REST API or UCI to start a game, make moves, and receive bot moves
Pre Condition	API/UCI connection is established, game parameters provided
Success Scenario	Game is created, moves are processed, and bot responses are returned successfully
Failure Scenario	API request fails, invalid move submitted, or connection drops

Table 3.3: Use Case Scenario for Viewing Game Status

Use-Case Identifier	UC3: View Game Status
Primary Actor	User, External System
Secondary Actor	X
Description	User or external system checks the current game state, including move history and result (e.g., ongoing, checkmate, draw)
Pre Condition	Game is in progress or completed, user/system is authenticated
Success Scenario	Game status and history display correctly
Failure Scenario	Status data is missing or incorrectly displayed

The use case scenarios (Tables 3.1, 3.2, 3.3) detail the interactions and outcomes for playing against the bot, managing games via an external system, and viewing game status.

Non-functional Requirements

Non-functional requirements of a system are a set of applications that describe the system's operational capabilities and constraints. These are basically the requirements that outline how well a system should operate. The following are the non-functional requirements of the UCHES:

- Respond to user moves within 1 second at depth 2 on standard hardware.
- Provide a responsive and intuitive user interface on desktop and tablet devices.
- Ensure secure data storage and transmission using Spring Security and PostgreSQL.

3.1.2 Feasibility Analysis

Feasibility analysis is a part of system analysis carried out to confirm whether the system developed is actually feasible or not. Following the feasibility analysis, the project is worked on:

Technical Feasibility

UCHESS leverages open-source technologies, including Angular, Chess.js, Angular Material, Spring Boot, and PostgreSQL, all compatible with standard development environments. PlantUML is used for creating UML diagrams, ensuring accurate system modeling. The Negamax algorithm with AlphaBetaPruning is computationally feasible for depth 2+ on typical hardware.

Operational Feasibility

The system is designed for ease of use, requiring minimal training for users familiar with chess. Role-based authentication simplifies access control, with intuitive Angular Material UI components that enhance the user experience.

Economic Feasibility

Using open-source tools and cloud hosting (e.g., Heroku for deployment) minimizes development and operational costs. No proprietary software or licenses are required, making the project economically viable.

3.1.3 Object Modelling: Object & Class Diagram

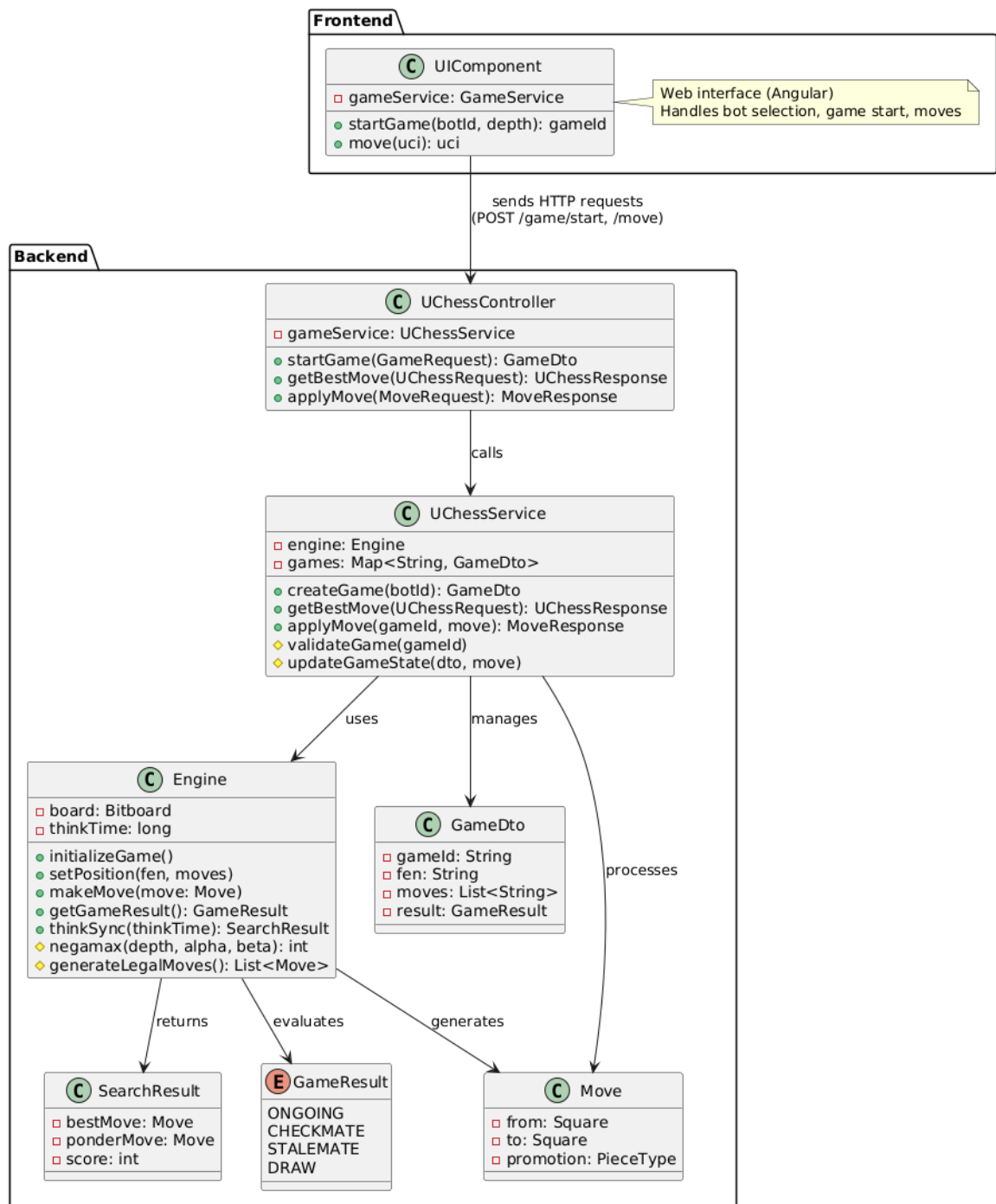


Figure 3.2: Class Diagram for UCHESS

Figure 3.2 shows the relationships between classes, with Game aggregating Board and Bot.

3.1.4 Dynamic Modelling: State & Sequence Diagram

The state diagram models the game lifecycle, including states like Initial, Playing, and Ended.

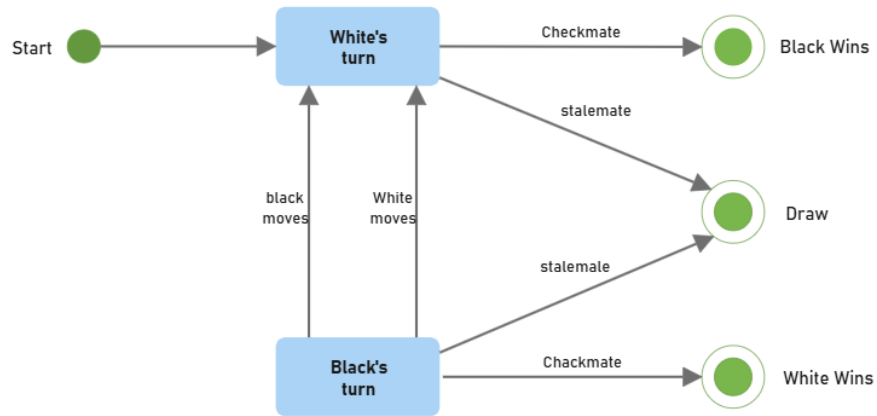


Figure 3.3: State Diagram for Game

The sequence diagram illustrates move execution, showing interactions between the user, front-end (Angular), back-end (Spring Boot), and bot.

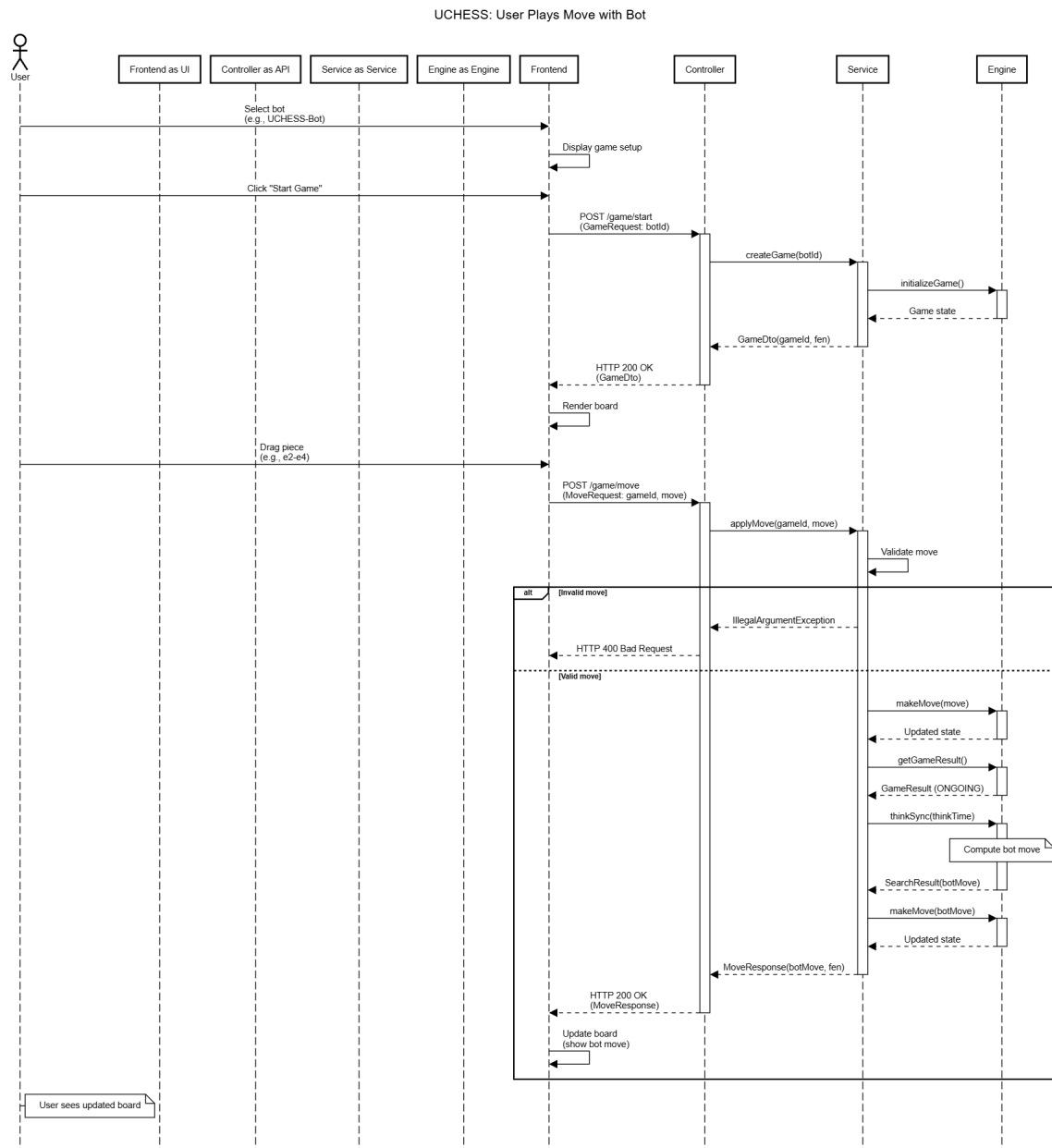


Figure 3.4: Sequence Diagram

3.1.5 Process Modelling: Activity Diagram

The activity diagram outlines the game flow, including starting a game, making a user move, the bot's response, and game termination.

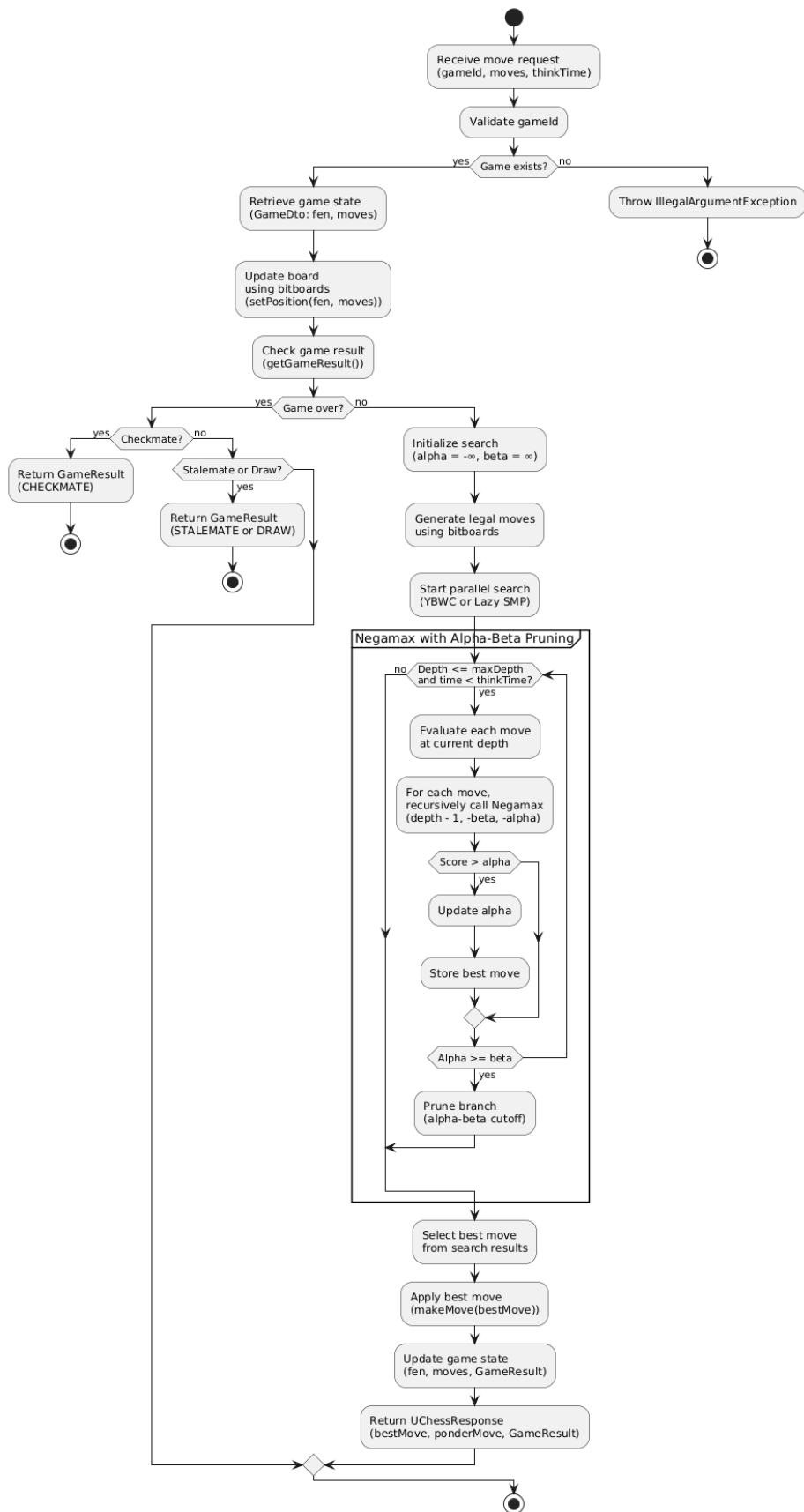


Figure 3.5: Activity Diagram for Game Flow

3.2 System Design

3.2.1 Refinement of Classes and Object

Classes are refined to include specific methods:

- Game: startGame(), endGame(), validateMove().
- Board: generateMoves(), updateBitboard().
- Bot: evaluatePosition(), negamax().
- AuthService: authenticateUser(), changePassword().

3.2.2 Component Diagram

The component diagram shows the modular structure of the system, including the Angular front end, the Spring Boot back end, the PostgreSQL database, and the Chess.js library.

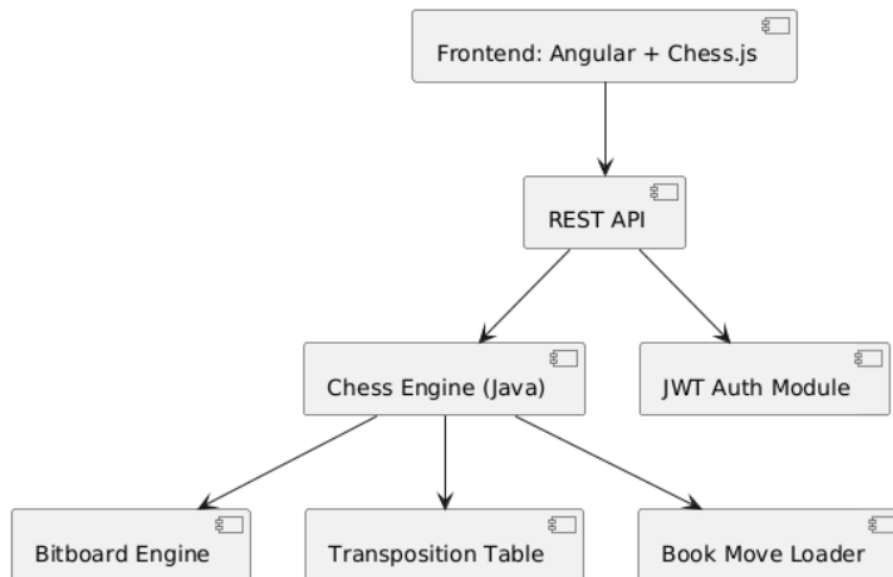


Figure 3.6: Component Diagram for UCHESS

3.2.3 Deployment Diagram

The deployment diagram illustrates the architecture of the system, with the front end hosted on a Web server, the back end on a Spring Boot server, and data stored in a PostgreSQL database.

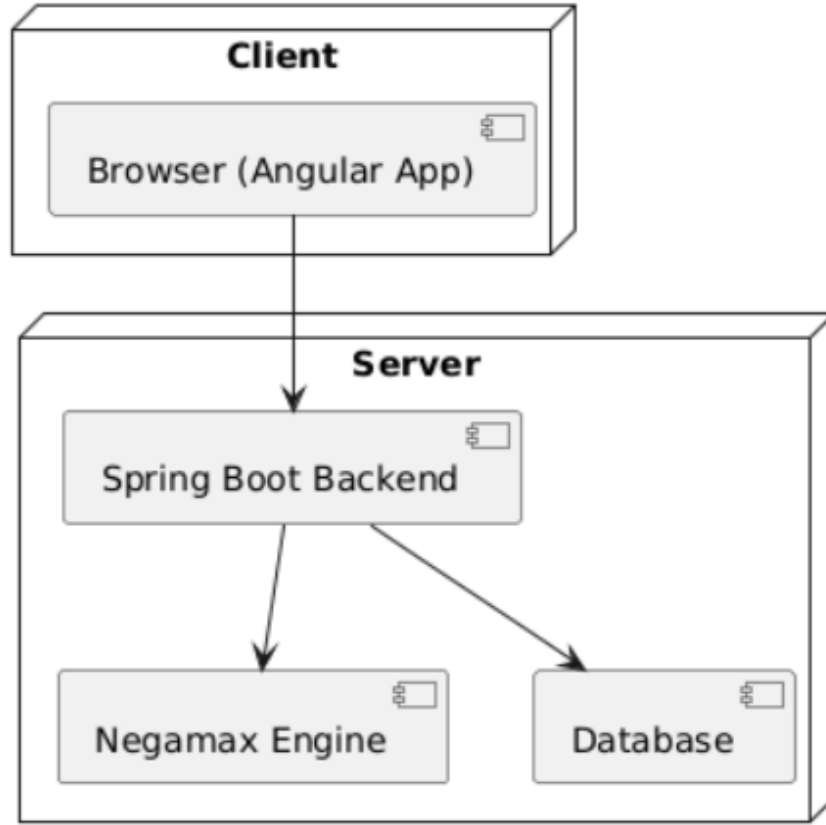


Figure 3.7: Deployment Diagram for UCHESS

3.2.4 Algorithm Details

The Negamax algorithm with AlphaBetaPruning is implemented as follows:

$$\text{Negamax}(\text{node}, \text{depth}, \alpha, \beta) = \max(-\text{Negamax}(\text{child}, \text{depth} - 1, -\beta, -\alpha))$$

Bitboards use 64-bit integers to represent piece positions, with bit shifting for move generation. Zobrist hashing generates unique keys for board states, stored in transposition tables to avoid redundant evaluations. The algorithm is optimized for depth 2+, ensuring responsiveness.

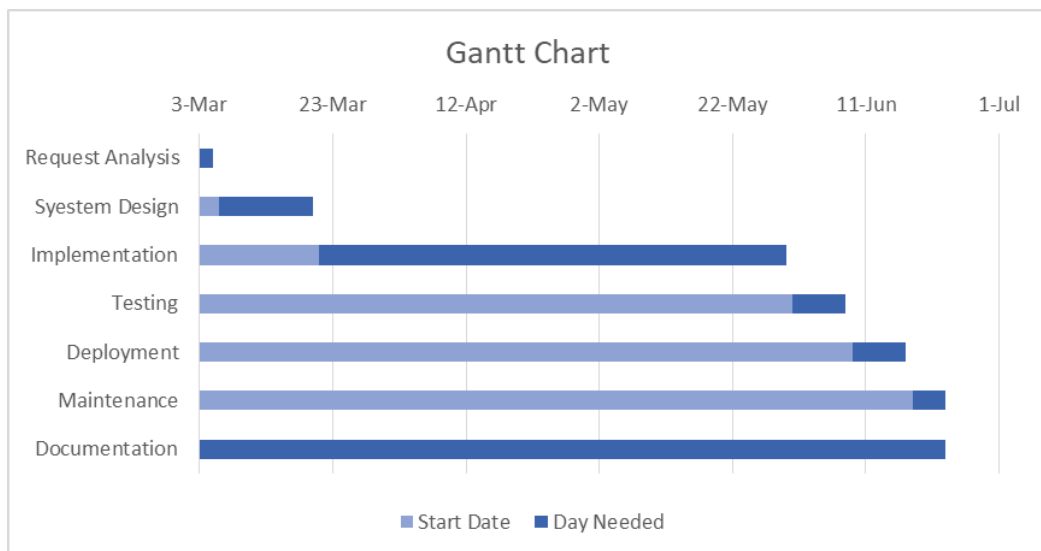
Negamax with Alpha-Beta Pruning

```
function negamax(node, depth,  $\alpha$ ,  $\beta$ , color):  
    if depth == 0 or node is terminal:  
        return color * evaluate(node)  
  
    maxEval =  $-\infty$   
    for each child in generate_moves(node):  
        eval = -negamax(child, depth - 1,  $-\beta$ ,  $-\alpha$ , -color)  
        maxEval = max(maxEval, eval)  
         $\alpha$  = max( $\alpha$ , eval)  
        if  $\alpha$  >=  $\beta$ :  
            break // Beta cutoff  
    return maxEval
```

3.3 Gantt Chart

The project timeline is illustrated in the Gantt chart below, spanning the sixth semester (6 weeks, starting March 2025).

Figure 3.8: Project Timeline



Chapter 4: Implementation and Testing

4.1 Implementation

4.1.1 Tools Used

The UCHESS project was developed using a combination of modern tools and technologies to ensure efficiency and scalability:

Table 4.1: Tools and Technologies Used in UCHESS Project

Category	Tools and Description
CASE Tools	PlantUML for creating UML diagrams (Class, Sequence, Activity) for system modeling and design.
Frontend	Angular for SPA development, Chess.js for rules and validation, Angular Material for responsive UI.
Backend	Spring Boot for RESTful APIs and business logic, PostgreSQL for data persistence.
Database	PostgreSQL
Testing	JUnit for unit testing, Postman for API testing.
Development Environment	IntelliJ IDEA and WebStorm for coding, Git for version control.
Documentation	Overleaf (using latex) for documents.
Diagram	PlantUML, mermaid, draw.io for figures
Version Control	Git and Github for version control

4.1.2 Implementation Details of Modules

The UCHESS system is divided into several key modules, each addressing specific functionality:

- **Game Module:** Manages the game state and rules using Chess.js. It initializes the board, validates user moves, and triggers bot responses. The module interfaces with the Board and Bot modules to maintain game flow.
- **Board Module:** Implements bitboards using 64-bit integers to represent piece positions. Bit shifting operations generate legal moves efficiently, reducing computational overhead.

- **Bot Module:** Implements the Negamax algorithm with AlphaBetaPruning for move evaluation. Zobrist hashing generates unique keys for board positions, stored in transposition tables to cache evaluations and avoid redundant computations. The bot supports adjustable difficulty starting at depth 2.
- **AuthService Module:** Handles role-based authentication using Spring Security. JSON Web Tokens (JWT) are used to secure user sessions, with roles (e.g., User, Admin) controlling access to game features.
- **Frontend Module:** Built with Angular, it renders the chessboard using Chess.js and Angular Material components (e.g., mat-board, mat-button). It communicates with the backend via REST APIs to send user moves and receive bot responses.
- **Backend Module:** Developed with Spring Boot, it exposes REST endpoints (e.g., /api/game/move, /api/auth/login) to handle game logic and authentication. PostgreSQL stores user data and game history.

The implementation prioritizes modularity, with clear separation between frontend and backend. The bot's performance is optimized through bitboards and transposition tables, ensuring responsiveness at depth 2 and above.

4.2 Testing

Testing was conducted to ensure the reliability and correctness of UCHESS's components and overall system.

4.2.1 Test Cases for Unit Testing

Unit tests were designed to validate individual modules. Table 4.2 lists key test cases.

Table 4.2: Unit Test Cases

Test Case	Description	Input	Expected Output
TC1	Validate move generation	Initial board position	List of legal moves
TC2	Test Negamax at depth 2	Sample board position	Best move within 2 seconds
TC3	Verify Zobrist hashing	Board position	Unique hash key
TC4	Test user authentication	Valid credentials	JWT token

4.2.2 Test Cases for System Testing

System tests verified the integration of all modules. Table 4.3 lists key test cases.

Table 4.3: System Test Cases

Test Case	Description	Input	Expected Output
TC1	User login and session	Valid credentials	Successful login, game access
TC2	Gameplay flow	User move (e.g., e4)	Bot responds (e.g., e5)
TC3	API communication	Move request via REST	Valid bot response
TC4	UI responsiveness	Different screen sizes	Consistent chessboard display

System tests were conducted using Postman for API endpoints and manual testing for UI responsiveness across devices. All test cases passed, confirming the system's functionality and performance.

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

The UCHESS project successfully met its core objectives, delivering a functional and optimized chess bot system. The primary goal was to create an AI opponent capable of playing against human users by implementing the Minimax algorithm with Alpha-Beta Pruning. This ensured that the bot could make competitive and strategic moves while maintaining efficient decision-making even at deeper search levels. The system also incorporated complete move validation and chess rule enforcement, guaranteeing that all gameplay adhered strictly to official chess standards.

To improve the performance of the chess engine, the project employed advanced data structures such as bitboards, Zobrist hashing, and transposition tables. These optimizations significantly reduced computation time by enabling faster move generation, accurate position hashing, and the elimination of redundant calculations. As a result, the bot delivered consistent and responsive performance suitable for various difficulty levels.

Another key achievement was the development of an interactive and user-friendly interface. Built using Angular alongside Chess.js and Angular Material, the interface allowed players to easily engage with the chess bot, offering smooth animations, intuitive controls, and a visually appealing board layout. This focus on usability ensured that the system could cater to both beginners and intermediate chess players, providing an enjoyable playing experience.

Overall, UCHESS achieved its objectives by combining strong AI algorithms, optimized performance techniques, and a polished frontend interface, creating a robust and engaging chess-playing platform.

5.2 Lesson Learnt/Outcome

The development of UCHESS enhanced several technical and project management skills:

- Implementing bitboards and Zobrist hashing improved the understanding of efficient data structures for chess engines, overcoming challenges in optimizing move generation and evaluation.
- Building the frontend with Angular and Angular Material, and the backend with Spring Boot, deepened expertise in modern web frameworks and RESTful API design.
- Conducting tests in Postman refined skills in ensuring system reliability and performance.
- Adhering to a structured timeline, from proposal to final defense, improved time management and iterative development practices under supervisor guidance.

Challenges, such as optimizing Minimax with Alpha-Beta Pruning for higher depths and

ensuring cross-browser UI compatibility, were addressed through iterative refinement and testing, resulting in a robust system.

5.3 Future Recommendations

To improve UCHESS in the future, the following changes can be made:

- Add Neural Network-based Evaluation (NNUE) to make move evaluation more accurate and improve gameplay strength without using too much processing power.
- Use WebSocket or GraphQL instead of REST APIs for real-time move updates, which will be helpful for adding multiplayer modes.
- Add more game modes like player vs. player and game analysis so that users can review their games with computer help.
- Create a mobile version using Angular's progressive web app feature so players can use UCHESS easily on their phones.
- Improve the bot's speed for higher search depths (for example, depth 4 or more) by using multithreading and GPU support to make it better for advanced players.

These updates will make UCHESS more useful, faster, and enjoyable for a wider range of users.

Chapter A: Screenshots

A.1 Screenshots

This section presents screenshots of the UCHESS web application, showcasing key user interfaces. These include the chessboard interface, login page for role-based authentication, and game history page, demonstrating the system's functionality and user experience.

A.1.1 Chessboard Interface

The chessboard interface is the primary view where users play against the bot. It displays the chessboard, rendered using Chess.js and Angular Material, with interactive squares for move selection.

Figure A.1: Chessboard Interface of UCHESS

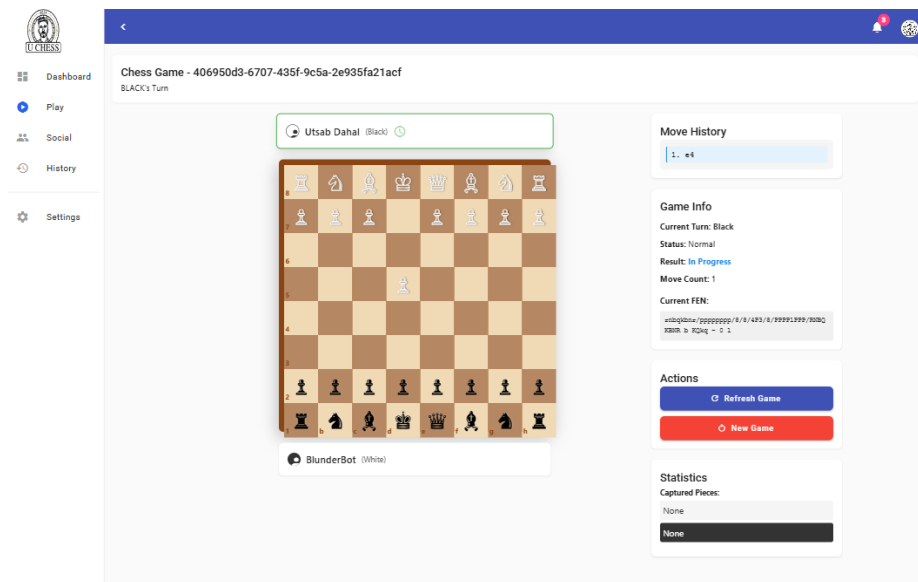
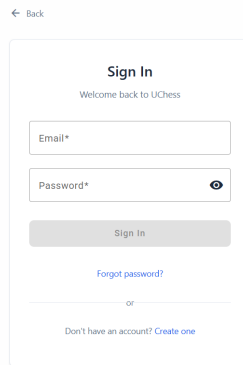


Figure A.1 shows a game in progress, with the board layout, pieces, and move indicators.

A.1.2 Login Page

The login page enables role-based authentication, implemented with Spring Security and Angular. Users enter credentials to access the game.

Figure A.2: Login Page of UCHESS



The login form is titled "Sign In" and includes a welcome message "Welcome back to UCheSS". It features input fields for "Email*" and "Password*" with a toggle for password visibility. A "Sign In" button is positioned below the password field. Links for "Forgot password?" and "Create one" are also present.

← Back

Sign In
Welcome back to UCheSS

Email*

Password* 

Sign In

[Forgot password?](#)

or

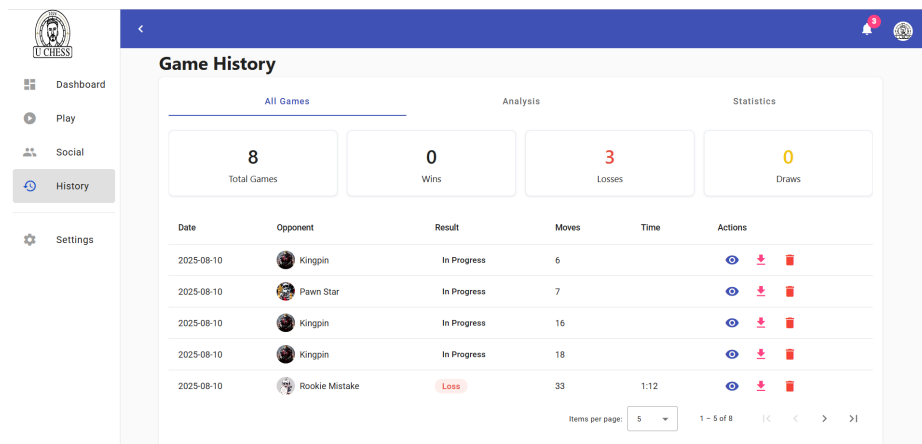
Don't have an account? [Create one](#)

Figure A.2 displays the login form, styled with Angular Material components for a responsive design.

A.1.3 Game History Page

The game history page allows users to view past games, including moves and outcomes, stored in the PostgreSQL database.

Figure A.3: Game History Page of UCHESS



The screenshot shows the "Game History" page with a sidebar menu and a main content area. The sidebar includes links for Dashboard, Play, Social, History (selected), and Settings. The main content area displays game statistics and a table of game history.

Game History

Navigation: All Games | Analysis | Statistics

Summary:

- 8 Total Games
- 0 Wins
- 3 Losses
- 0 Draws

Date	Opponent	Result	Moves	Time	Actions
2025-08-10	Kingpin	In Progress	6		  
2025-08-10	Pawn Star	In Progress	7		  
2025-08-10	Kingpin	In Progress	16		  
2025-08-10	Kingpin	In Progress	18		  
2025-08-10	Rookie Mistake	Loss	33	1:12	  

Items per page: 5 | 1 - 5 of 8 | < > >>

Figure A.3 shows a table of previous games, with options to review specific matches.

Chapter B:Code Snippets

B.1 Code Snippets

This section provides representative code snippets from the UCHESS project, demonstrating key functionalities such as the Negamax algorithm, frontend chessboard rendering, and backend API handling. The snippets include pseudo-code for the bot logic, TypeScript for the Angular frontend, and Java for the Spring Boot backend.

B.1.1 Negamax with AlphaBetaPruning

The following code illustrates the Negamax algorithm with AlphaBetaPruning, used for the bot's move evaluation.

```
1  @Override
2      public SearchResult negamax(Duration duration) {
3
4          start = Instant.now();
5          timeout = start.plus(duration);
6          nodes = 0;
7          currentDepth = 1;
8          bestMove = null;
9          bestMoveCurrentDepth = null;
10         bestEval = 0;
11         bestEvalCurrentDepth = 0;
12         cancelled = false;
13
14         int alpha = Integer.MIN_VALUE + 1;
15         int beta = Integer.MAX_VALUE - 1;
16         int retryMultiplier = 0;
17         SearchResult result = null;
18
19         while (!isCancelled() &&
20             currentDepth < config.getMaxDepth()) {
21             // Reset variables for the current depth iteration
22             bestMoveCurrentDepth = null;
23             bestEvalCurrentDepth = 0;
24
25             // Perform alpha-beta search for the current depth
26             int eval = search(
27                 currentDepth, 0, alpha, beta, true
28             );
29
```

```

30 // Update the best move and evaluation if a
31 // better move is found
32 if (bestMoveCurrentDepth != null) {
33     bestMove = bestMoveCurrentDepth;
34     bestEval = bestEvalCurrentDepth;
35     result = buildResult();
36     threadManager.handleSearchResult(result);
37 }
38
39 // Check if search is cancelled
40 // or a checkmate is found
41 if (
42     isCancelled()
43     ||
44     isCheckmateFoundAtCurrentDepth(currentDepth)
45 ) {
46     break;
47 }
48
49 // Adjust the aspiration window in case the score
50 // fell outside the current window
51 if (eval <= alpha) {
52     // If score <= alpha, re-search with an
53     // expanded aspiration window
54     retryMultiplier++;
55     alpha -= config.getAspFailMargin()
56         * retryMultiplier;
57     beta += config.getAspMargin();
58     continue;
59 }
60 if (eval >= beta) {
61     // If score >= beta, re-search with
62     // an expanded aspiration window
63     retryMultiplier++;
64     beta += config.getAspFailMargin()
65         * retryMultiplier;
66     alpha -= config.getAspMargin();
67     continue;
68 }
69
70 alpha = eval - config.getAspMargin();

```

```

71         beta = eval + config.getAspMargin();
72
73         // Increment depth and retry multiplier for
74         // next iteration
75         retryMultiplier = 0;
76         currentDepth++;
77     }
78
79     // If no move is found within the time limit,
80     // choose the first available move
81     if (result == null) {
82         System.out.println(
83             "Time expired before a move was found!"
84         );
85         bestMove = moveGenerator.generateMoves(board)
86             .getFirst();
87         result = buildResult();
88     }
89
90     // Clear move ordering cache and return the
91     // search result
92     moveOrderer.clear();
93     return result;
94
95 }

```

Listing B.1: Negamax Search Implementation

The algorithm uses bitboards for move generation and Zobrist hashing to cache evaluations, optimizing performance for depth 2 and above.

B.1.2 Angular Chessboard Component

The following TypeScript snippet shows an Angular component for rendering the chessboard and handling user moves, using Chess.js for game logic.

Listing B.2: Angular Chessboard Component

```

1 import { Component } from '@angular/core';
2 import * as Chess from 'chess.js';
3
4 @Component({
5     selector: 'app-chessboard',
6     template: '

```

```

7     <div class="chessboard" (click)="makeMove($event)">
8         <mat-grid-list cols="8" rowHeight="50px">
9             <mat-grid-tile *ngFor="let_square_of_board"
10                [ngClass]="squareClass(square)">
11                 {{ square.piece }}
12             </mat-grid-tile>
13         </mat-grid-list>
14     </div>
15     '
16 })
17 export class ChessboardComponent {
18     private chess = new Chess();
19     board: any[] = [];
20
21     constructor() {
22         this.updateBoard();
23     }
24
25     makeMove(event: any): void {
26         const move = this.chess.move(event.source);
27         if (move) {
28             this.updateBoard();
29             this.sendMoveToBackend(move);
30         }
31     }
32
33     updateBoard(): void {
34         this.board = this.chess.board();
35     }
36
37     async sendMoveToBackend(move: any): Promise<void> {
38         const response = await fetch('/api/game/move', {
39             method: 'POST',
40             body: JSON.stringify(move)
41         });
42         const botMove = await response.json();
43         this.chess.move(botMove);
44         this.updateBoard();
45     }
46 }

```

This component renders the chessboard using Angular Material's grid list and communicates user moves to the backend via REST APIs.

B.1.3 Spring Boot REST Controller

The following Java snippet demonstrates a Spring Boot REST controller for handling game moves and authentication.

```
1 package com.uchess.controller;
2
3 import com.uchess.service.GameService;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.security.access.prepost.PreAuthorize;
6 import org.springframework.web.bind.annotation.*;
7
8 @RestController
9 @RequestMapping("/api/game")
10 public class GameController {
11
12     @Autowired
13     private GameService gameService;
14
15     @PostMapping("/move")
16     @PreAuthorize("hasRole('USER')")
17     public String makeMove(@RequestBody String move) {
18         return gameService.processMove(move);
19     }
20
21     @PostMapping("/auth/login")
22     public String login(@RequestBody AuthRequest request) {
23         return gameService.authenticateUser(
24             request.getUsername(), request.getPassword()
25         );
26     }
27 }
```

Listing B.3: SpringBoot Rest Controller

This controller handles move requests and user authentication, using Spring Security for role-based access control and JWT tokens for session management.

References

- [1] A. Säfsström and S. Yllmark, “A comparative study of parallel search techniques in a chess engine: Young brothers wait concept or lazy smp?,” 2025.
- [2] D. Rasmussen, “Parallel chess searching and bitboards,” Master’s thesis, Technical University of Denmark, DTU, DK-2800 Kgs. Lyngby, Denmark, 2004.
- [3] A. Kothari, V. R. N. S. Nikhil, H. Reddy, N. Rohit, and M. Khanna, “Efficiency through algorithms: Chess engine without machine learning,” in *AIP Conference Proceedings*, vol. 3237, p. 020001, AIP Publishing LLC, February 2025.